

Professor comments, 25 May 2026

This page is added for side-by-side review. The original thesis pages follow after this review sheet; matching anchors are lightly highlighted in the document.

ID	Comment	Anchor
C1	Gap between the previoud paragraph and this sentence. It needs to explicitey explain that a chatbot can be a solution for the problems pointed out in the previous paragraph. Then you can state the requirements for chatbot design	key point for the chatbot design
C2	Need to be more specific. Small for what (GPU memory , system memory, parameter size?) How much the model to be small? They should be clearly specified with reasons.	small enough to run on a single GPU server
C3	"smallest" should not be the goal, but small enough should be.	smallest local model configuration
C4	recommended to present as a table, with explanations for each source (i.e. what course records contain, what is e-handbook) so that readers other than ICU can understand.	ICU university corpus
C5	This benchmark can be a part of the answer for the RQ1, so how to construct the questions and why they are enough should be explained to answer RQ1	ICU Bench 30

ICU Wiki: A Case Study in Reliable Local RAG for University QA

261768 ZHANG Yizhou

Abstract

University information is often fragmented across multiple websites and hidden behind layers of menus, making it difficult for students to find expected answers efficiently. General-purpose LLM chatbots are widely used and good for question answering, but they usually do not have access to institution-internal data that is access-controlled. Retrieval-augmented generation (RAG) is a natural approach for answering questions from a custom document corpus, but using API-based models behind such a system can increase long-term cost and may raise data-governance concerns, especially for a non-profit university project.

This thesis presents and evaluates ICU Wiki, a locally hostable RAG-based chatbot for university question answering using locally runnable small language models. The central question is how to define and identify the smallest reliable local configuration, including both the language model and the retrieval-and-orchestration layer around it. We construct a cleaned ICU corpus and evaluate the system on [ICU Bench 30](#), a 30-question benchmark designed around major question categories in the ICU corpus. We test six models under two answer modes and generate 360 answers, which are evaluated using a G-Eval-inspired ([Liu et al., 2023](#)) two-judge method across answer acceptance, readability, citation usefulness, and over-answering risk.

The main finding is that average score alone is not sufficient for model selection: smaller models can receive relatively high average scores while still producing unusable or blocking failures under evidence-based QA requirements. Under the proposed reliability criterion, Gemma 4 E4B becomes the smallest observed reliable candidate, while Qwen3.5 9B provides the strongest larger alternative. Overall, this thesis contributes ICU Wiki as a case study of how to build and evaluate an evidence-based local RAG system for university question answering on a fragmented, multilingual institutional corpus, using models small enough to run on local hardware.

1 Introduction

1.1 University QA as an Evidence-Based Problem

University information is often fragmented across multiple systems and hidden behind layers of menus, and ICU is one example of this. We have sites like the e-handbook, the ICU Portal, public web pages, the ICU map, course and syllabus pages, and so on. Students need to go through multiple layers of menus to find the information they need, which is sometimes inefficient. We currently have a keyword-match search function inside the e-handbook, but students often come up with questions in a natural language format rather than just a single keyword. Also, students often need to combine information from several sources instead of just one. However, the current search function inside the handbook is not sufficient for that and requires them to navigate through multiple sources to get the final answer.

Students’ questions, and the answers they receive, can affect real decision-making, for example, course registration, graduation requirements, senior thesis policy, and so on. A wrong answer could lead students to take wrong actions or make wrong decisions, such as missing a deadline, misusing generative AI tools like ChatGPT, or going to the wrong location when trying to meet with their professor. Therefore, the system cannot only pursue fluency; it also needs to be evidence-based and needs to refuse to answer when the evidence is not sufficient. In this setting, a wrong answer is not only unhelpful, but can also become misleading guidance and thus cause a much more serious result.

Therefore, the **key point for the chatbot design** is not only whether the system can retrieve relevant information and give a fluent answer, but also whether it can distinguish between sufficient and insufficient evidence and decide whether it should answer the student’s question or not. It should only answer when the retrieved institutional evidence supports the answer, and otherwise it should clearly refuse and state that the evidence is insufficient to generate the final answer.

1.2 Local deployment constraints

In terms of deployment, an API-based service could create excessive long-term cost and potential data-governance risks, especially when the imported e-handbook data contains internal school information that is meant to be accessible only to ICU students. Therefore, we aim to design the system to be locally deployable. This can avoid long-term API costs and keep the data inside the school environment to reduce possible leakage risks. We therefore aim to reach a certain level of system efficiency, where the model is **small enough to run on a single GPU server**. However, before running this evaluation, whether a small model would be capable enough for this task was unknown.

Smaller models are easier to deploy and may have shorter latency, but there are also important downsides. For example, they may be more likely to over-answer questions and miss key conditions inside the retrieved evidence. They may also be less capable of deciding whether the retrieved evidence is sufficient or insufficient. Their output format can also be less consistent than that of larger models. For a student-facing Q&A use case, these problems are dangerous because the answer may affect real student decisions.

1.3 Research questions

Based on the evidence-safety problem and the answer uncertainty around small, locally hosted models discussed above, this thesis uses ICU Wiki as a case study to examine how reliable a locally hosted RAG system can be for student-facing university question answering, how reliability should be defined, and how the smallest reliable local configuration can be identified. This thesis uses the following main research question and three subquestions.

Main research question. How can we build and evaluate a local RAG system for student-facing university question answering, and identify the **smallest local model configuration** that is reliable enough for this task?

RQ1. What should count as a reliable answer in student-facing university QA, and are standard RAG evaluation scores enough to measure it?

RQ2. How should the retrieval-and-orchestration layer be designed to help small local models

answer questions from a multilingual university corpus?

RQ3. Under this evaluation and system design, what is the smallest local model configuration that reaches the reliability requirement?

C4

1.4 Contributions

This thesis mainly makes three contributions. First, it presents ICU Wiki as a case-study reference for building a local RAG-based chatbot over a fragmented, multilingual, and multi-format university or institutional corpus. Although the system is built for ICU, a similar setting can be shared by universities and institutions whose information is distributed across multiple domains and stored in various formats.

Second, it proposes a retrieval-and-orchestration layer for this setting, showing how small local language models can be supported by system-side modules and identifying which modules contribute most under small-local-model constraints.

Third, it proposes an evaluation logic for small language models in local RAG-based systems. Instead of fully trusting scores from mainstream RAG evaluation frameworks, it identifies constraints of those evaluation metrics and proposes additional dimensions that need to be evaluated separately or in layers to understand the real reliability and usability of small language models in this setting.

2 Background and Related Work

3 Task and University Corpus

3.1 ICU university corpus

As mentioned above, the ICU corpus is fragmented, multilingual, and multi-format. It contains ICU-related institutional knowledge spread across multiple domains and represented in different formats. We mainly prepare the data from five source groups: course records, ICU e-handbook pages, public web pages under ICU-related domains, portal-derived items, and PDF documents discovered from those pages or links. We treat PDF documents as a separate source group because many policy or guide materials are not written directly inside HTML pages but are attached as downloadable files.

Our goal is to include as much data from ICU-related official or linked sources as possible, so that the system can cover a wider range of student questions.

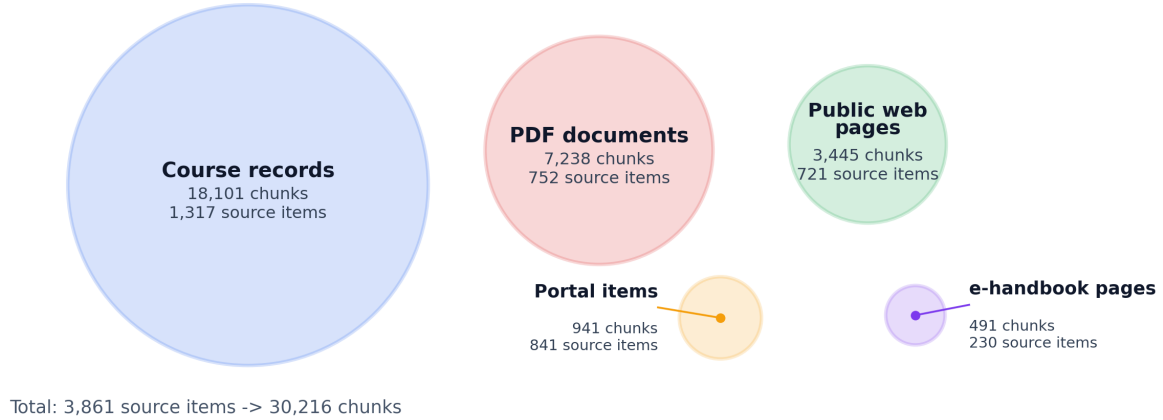


Figure 1: ICU corpus composition by generated chunk count.

3.2 Corpus construction

We collect raw data from the sources mentioned above. However, raw web data contains many headers, boilerplate sections, navigation texts, and repetitive text blocks that appear across web pages because of the HTML structure. Therefore, we normalize the collected data by keeping useful metadata and removing unnecessary text.

We then apply source-aware chunking. Instead of using one fixed chunking strategy for every source, we use different chunking strategies for different source types. For each chunk, we keep the citation and metadata, so that when the model references a chunk, the original source link can still be shown to users. Finally, we input all chunks into the embedding and retrieval-indexing pipeline.

3.3 ICU Bench 30

We prepared ICU Bench 30, which contains 30 questions that we believe cover the major categories inside our corpus data, with multiple languages and a certain difficulty threshold. We provided each question and its retrieved chunks to six models, including the Qwen3.5 family from 0.8B to 9B and the Gemma 4 2B and 4B models. Each model generated answers in two modes: single-shot and agentic. As a result, we received 360 answer pairs.

4 System Design

4.1 System overview

4.2 Retrieval-and-orchestration layer

4.3 Retriever design

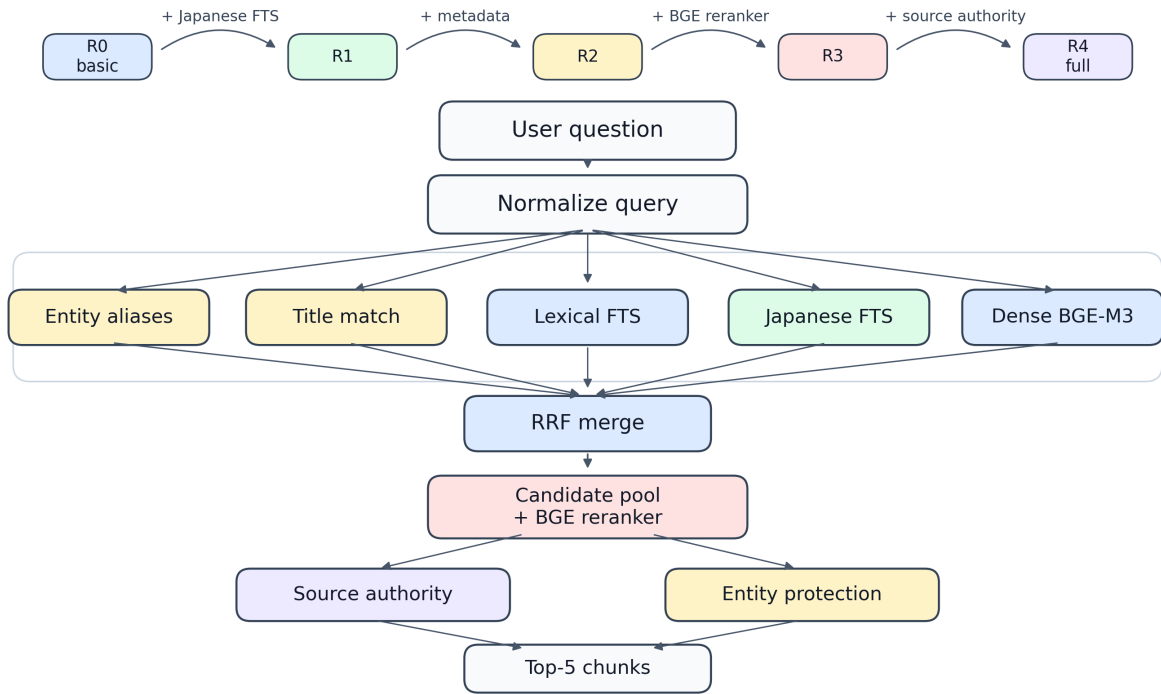


Figure 2: Retriever iteration.

Implementing the retriever on the ICU corpus was challenging because the corpus contains multiple categories, document types, and two languages. First, we tried the simplest hybrid retrieval method, which combines keyword retrieval and dense retrieval based on the BGE-M3 embedding model, the same model we used to embed all the chunks in the ICU corpus. This worked for some basic natural-language search, but it was unreliable for questions in Japanese and for questions that contained specific terminology related to ICU or technical terms, including professors’ names, ICU-specific terminology, and numbers.

We analyzed the bad cases and found that the simple text-search configuration from PostgreSQL worked well for English sentences because English words are separated by spaces. However, for Japanese sentences, it did not know how to separate the sentence into proper words. Therefore, for keyword matching, especially when a query contained a very specific keyword that gave a strong hint about which corpus area it might relate to, the result was poor. One example was 楠木先生が担当している授業は何ですか？ (“Which classes does Professor Kaburagi teach?”). The English or romanized version could sometimes work better, but the Japanese version was much less reliable before the Japanese retrieval improvements.

Before using the Japanese tokenizer, the system could only process the sentence as a whole or use weak CJK lexical separators:

楠木先生が担当している授業は何ですか？

As a whole sentence: 楠木先生が担当している授業は何ですか？

With CJK separators: 楠木 / 木先 / 先生 / 生が / が担 / 担当 / 当し / して / てい / いる / る授 / 授業

With Janome tokenizer: 楠木 / 先生 / 担当 / 授業

We therefore upgraded the lexical side with PostgreSQL FTS and an additional Janome Japanese tokenizer, so that the system could split Japanese question sentences into more proper word units and produce better keyword-matching results. For certain terminology matching, because specific terms can provide a strong hint during retrieval, we also built an alias table from course and person metadata. This alias table improves exact matching for data chunks linked to certain entities, especially for teacher and course-name-related lookups.

We then expanded the candidate pool and added the BGE reranker. This allowed the system to first collect a broader set of candidate chunks and then rerank them by comparing the question and candidate passages more directly. After that, we found another type of bad case. For questions such as 卒論に ChatGPT を使っても大丈夫ですか？ (“Is it okay to use ChatGPT for a senior thesis?”), the retriever tended to retrieve chunks from newsletters or event-like pages. Those chunks surely contained related words and related content, but for questions related to policies, we wanted evidence from more authoritative policy-related materials, such as the e-handbook.

Therefore, we added source-authority scoring. Instead of only asking whether a chunk is relevant to the query, this scoring also considers the source type and the broad query category. For example, policy and procedure questions should prefer e-handbook, official web pages, or portal notices, while course-specific questions should prefer course records. This shifted the retriever from finding merely relevant chunks toward selecting evidence that is more appropriate for a specific student-facing question.

Figure 3 shows a bad case from an earlier runtime. Here we can see that Japanese and English queries can retrieve different evidence under the same retriever, and policy questions need more

source authority rather than only keyword relevance. Although the meaning of the Japanese and English questions is almost the same, the Japanese request only retrieved newsletter-like content, while the English request retrieved sources from ICU official web pages and the e-handbook.

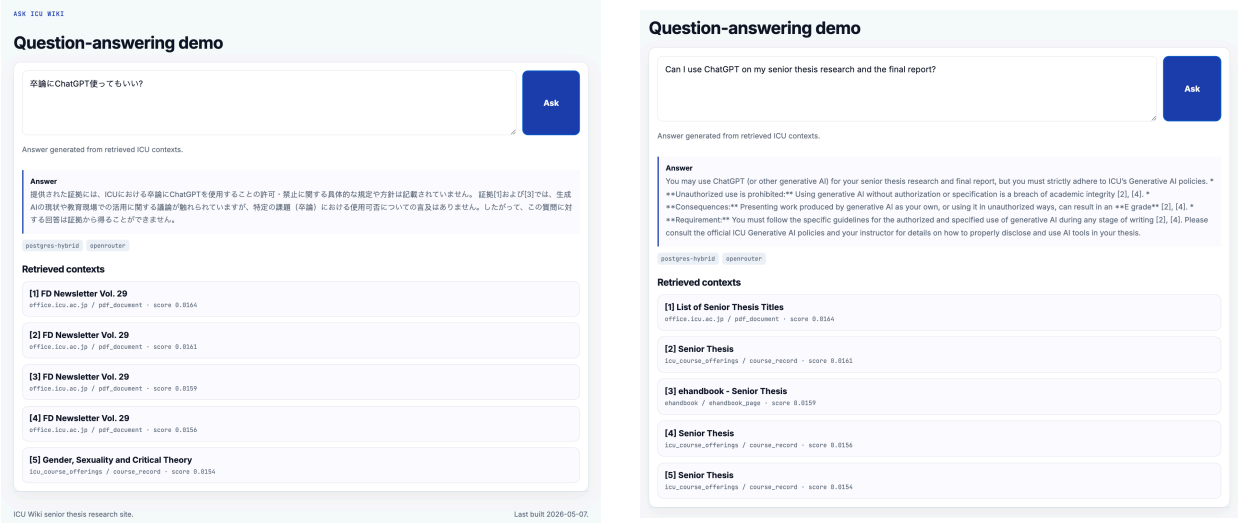


Figure 3: Qualitative comparison from the earlier runtime: Japanese and English ChatGPT thesis-policy queries retrieved different evidence under the same retriever.

4.4 Answer orchestration modes

4.4.1 Single-shot mode

In our system design, we use single-shot mode as the baseline. Under single-shot mode, each question passes through one fixed retriever and one fixed retrieval action, and the top-five evidence chunks are sent directly to the answer generator. As a result, it is easier to compare the difference between each model’s pure generation performance, because the retriever side is fixed and will not change because of an agent loop.

4.4.2 Multi-step retrieval mode

Considering recent progress in agentic retrieval methods, where the model can rewrite the query if the query is bad and perform multiple rounds of search if it thinks the evidence is not enough, we propose adding an agentic mode to the current baseline. Agentic mode can rewrite the input question into both English and Japanese variants, and it can perform multiple rounds of search if it thinks the retrieved evidence is not enough to generate the final answer. We limit the maximum number of search rounds to three to keep the system delay within a controllable scope.

However, agentic retrieval relies heavily on the model’s own performance. If the controller model behind it is not strong enough, it may misjudge evidence sufficiency. Under multiple rounds of retrieval, it may also bring in more irrelevant chunks, which can lead to a worse answer. Furthermore, agentic mode can significantly increase latency. Therefore, we cannot assume that agentic mode is always better, and careful comparison needs to be performed between models and generated answers under different questions. We then decide which mode is more suitable for our system

design.

5 Evaluation Methodology

5.1 What counts as a reliable answer?

Under the university QA situation, defining what should count as a reliable answer is important. We propose four criteria that both the model and the retriever need to meet:

- **Evidence-based.** The generated answer must be based on the retrieved evidence instead of being filled by the model’s own internal knowledge or unrelated evidence. This is extremely important, especially for questions related to school policy, registration deadlines, and course requirements.
- **Summarize instead of copied.** The answer needs to be rewritten in the model’s own words and understanding, and it needs to stay within a reasonable length constraint. It should not give a short answer and then directly copy and paste all five contexts into its generated answer, which we have observed in the 0.8B model’s generated results. The model needs to have a certain level of summarization ability to make the answer easy for students to read. Otherwise, there is no meaning in having the generator instead of only using the retriever itself.
- **Citation useful.** The generated answer needs to include useful citations. It needs to have citation marks, and each citation mark needs to be attached to the correct piece of generated text, so that students can check the original source while reading the generated answer.
- **No catastrophic failure.** We also want to avoid catastrophic failure, or bad cases, such as endless repetition loops from the model, truncation, gibberish, extreme hallucination, or answers based on insufficient evidence. If a model shows these catastrophic failure cases, even if it has a relatively high average score, we consider it not usable because it is not reliable enough.

5.2 Limits of standard RAG evaluation

At first, we used RAGAS as a mainstream evaluation method to evaluate the whole-system RAG performance. However, after running it across our six models, we found that the score differences were very small, making it difficult to determine the real model differences or set a threshold for deciding which model could be used for the actual system generation.

Furthermore, under these very close RAGAS scores, we found that the actual generated answers differed more than the scores suggested. For example, the 0.8B model’s readability and summarization ability were low. Even when it produced an answer that was correct enough and faithful to the retrieved context, it could still be unusable from a human perspective because it was not readable and was inconsistent in format. Therefore, we found that some metrics are not reflected by the RAGAS evaluation framework. The current RAGAS evaluation framework has certain constraints, especially for readability, model generation consistency, and bad-case detection.

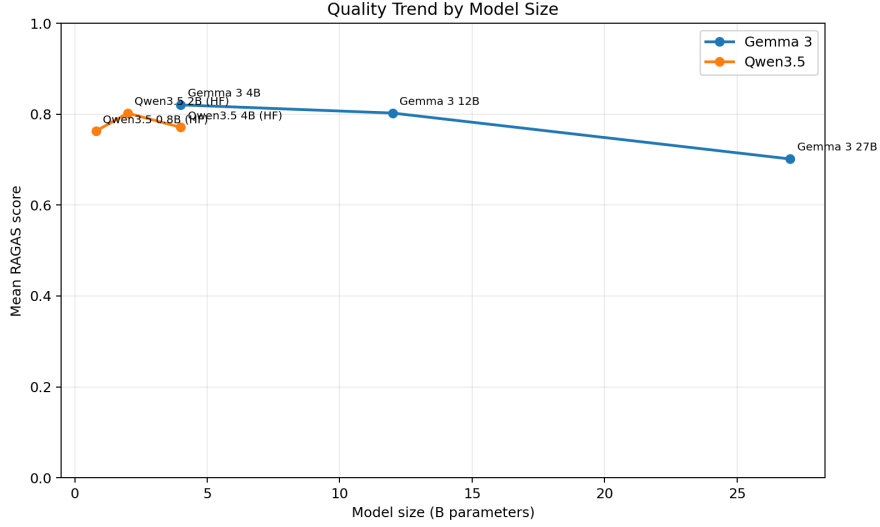


Figure 4: Early mean scores from RAGAS by model size, a diagnostic reference for the limits of RAGAS scoring.

5.3 G-Eval-inspired answer-quality evaluation

We then fed these answer pairs to GPT Pro and DeepSeek V4 Pro to let them blindly evaluate the answers. The judges gave scores from 0 to 2 on four dimensions: answer acceptance, readability, citation usefulness, and over-answering risk. To reduce possible evaluator bias from one model family, we intentionally chose two capable models from two different model families. The result shows that the stronger local models generally receive higher human-facing quality scores, but the difference between single-shot and agentic mode is not the same across model sizes.



Figure 5: ICU Bench 30 average scores under two judges across four evaluation dimensions.

5.4 Working reliability requirement

5.5 Bad and blocking cases

In this retriever ablation, we define a blocking case as an answer where `answer_acceptance` is 0, `human_quality_score` is below 0.50, or the judge notes/tags indicate repetitive or truncated output.

6 Experiments and Results

6.1 Model and mode comparison

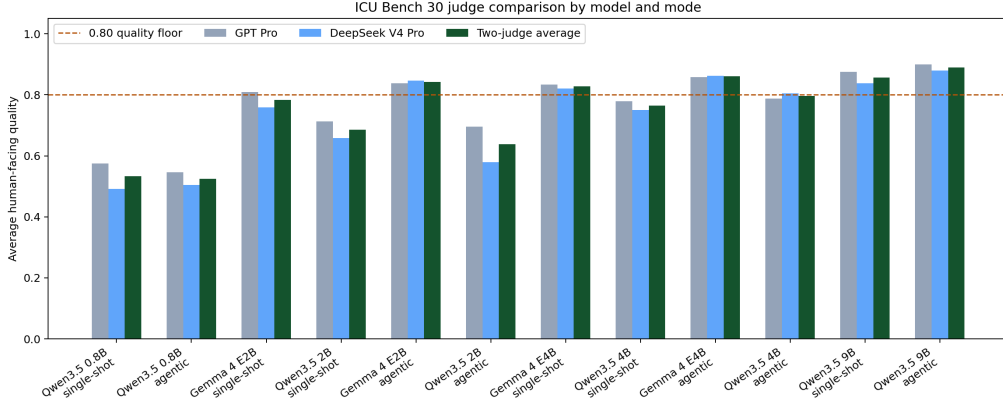


Figure 6: ICU Bench 30 judge comparison by model and answer mode.

We compared the quality scores produced by two SOTA-level judge models, GPT Pro and DeepSeek V4 Pro, and took their average to reduce single-judge model bias. Across the 12 model/mode conditions, GPT Pro gave a higher average score in nine conditions, while DeepSeek V4 Pro gave a higher average score in three conditions. This variability is the reason we use the average of the two judges instead of relying on a single model family.

If we set the working threshold to 0.8 on the two-judge average score, the conditions that reach this threshold are Gemma 4 E2B in agentic mode, Gemma 4 E4B in both single-shot and agentic modes, and Qwen3.5 9B in both single-shot and agentic modes.

6.2 Smallest reliable configuration

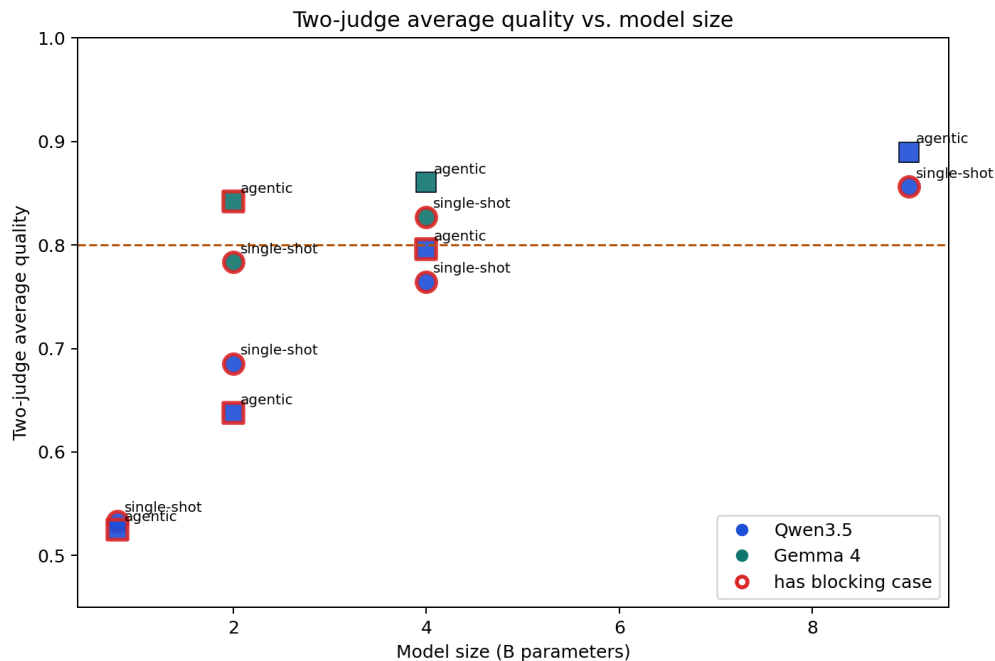


Figure 7: Average quality with blocking cases marked under two judges across model sizes and answer modes for the working-threshold screen.

The previous score-only comparison is not enough to clarify the practical difference between the models. Because catastrophic answer failures matter for an evidence-based chatbot, we add blocking-case analysis on top of the average score. After marking blocking cases across these model/mode conditions, we find that Gemma 4 E4B agentic mode and Qwen3.5 9B agentic mode are the only two conditions that are above the quality threshold while also having zero blocking cases.

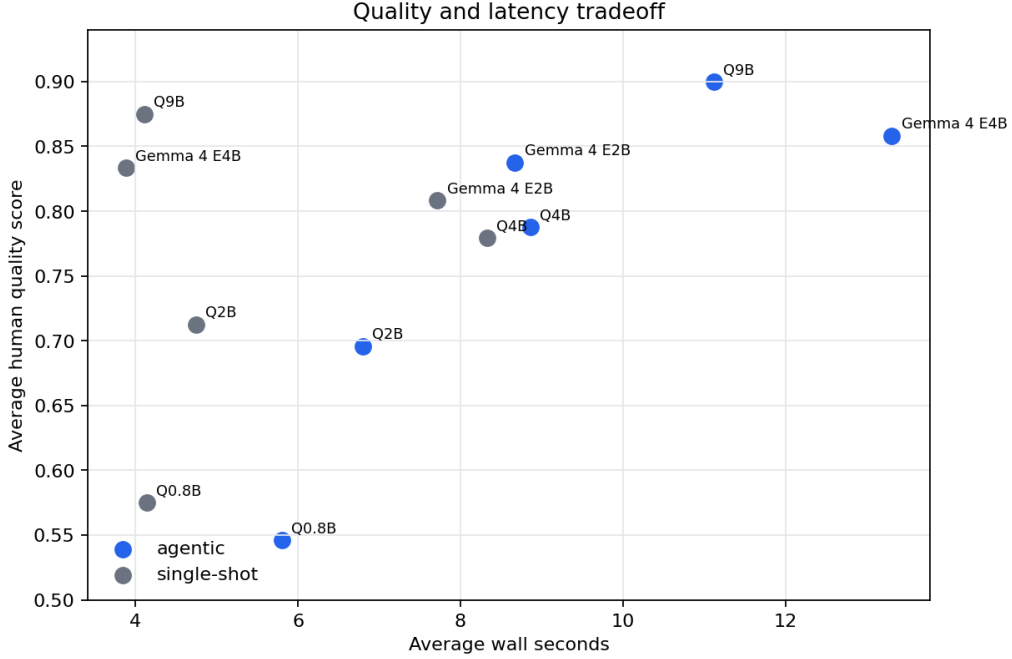


Figure 8: ICU Bench 30 quality-latency tradeoff by model and answer mode.

With Gemma 4 E4B and Qwen3.5 9B under agentic mode passing our reliability requirement, we then consider latency and hardware constraints. Qwen3.5 9B provides the strongest deployment choice if the hardware is strong enough and has enough GPU memory. Gemma 4 E4B, on the other hand, can fit into a tighter hardware constraint, although it showed higher latency in this run.

6.3 Retriever ablation

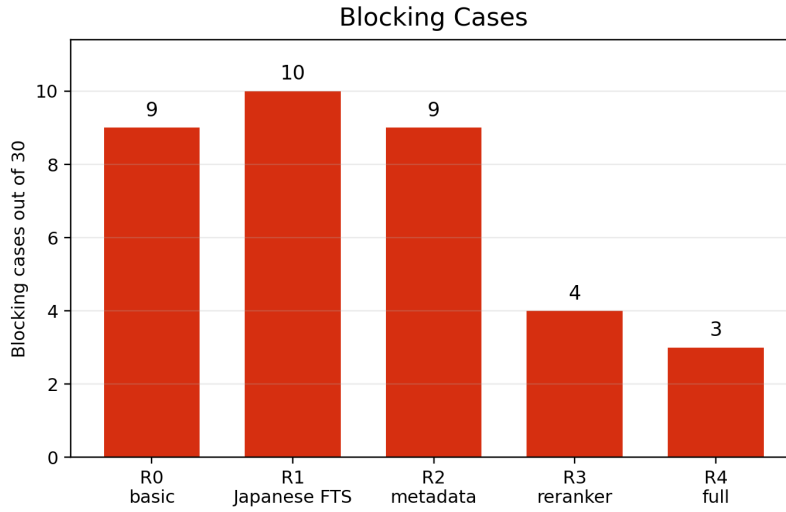


Figure 9: Blocking cases across retriever versions.

6.3.1 R0 to R1 (added Japanese FTS)

Blocking cases decreased

- s30_023: ICU に入学した後、必ず履修しなければならない科目は何ですか？どの公式ページを見るべきですか？
- s30_029: 楠木先生教哪些课？请只根据检索到的 ICU 课程资料回答。

Blocking cases increased

- s30_018: MIURA 先生って、どの日本語の授業を担当していますか？レベルも少し知りたいです。
- s30_025: 大学の卒論にチャット GPT を使っても大丈夫ですか？
- s30_027: 我可以在 ICU 的毕业论文里使用 ChatGPT 吗？如果资料不够，请明确告诉我。

6.3.2 R1 to R2 (added metadata/entity matching)

Blocking cases decreased

- s30_018: MIURA 先生って、どの日本語の授業を担当していますか？レベルも少し知りたいです。
- s30_028: 楠木先生が担当している授業は何ですか？

Blocking cases increased

- s30_007: I feel overwhelmed and want to talk to someone, but I am worried about privacy. Where can I find official information about counseling or student support at ICU?

6.3.3 R2 to R3 (added BGE reranker)

Blocking cases decreased

- s30_007: I feel overwhelmed and want to talk to someone, but I am worried about privacy. Where can I find official information about counseling or student support at ICU?
- s30_009: I am graduating from undergraduate school but continuing to ICU graduate school. What should I know about my IT account or services after graduation?
- s30_025: 大学の卒論にチャット GPT を使っても大丈夫ですか？
- s30_027: 我可以在 ICU 的毕业论文里使用 ChatGPT 吗？如果资料不够，请明确告诉我。
- s30_030: After I select courses on the registration website, how can I check whether my registration is actually complete?

Blocking cases increased

- None.

6.3.4 R3 to R4 (added source authority)

Blocking cases decreased

- s30_014: 履修登録について教育事務グループに確認したいです。連絡先や関連する公式ページはどこで確認できますか？

Blocking cases increased

- None.

6.4 Effect of answer orchestration mode

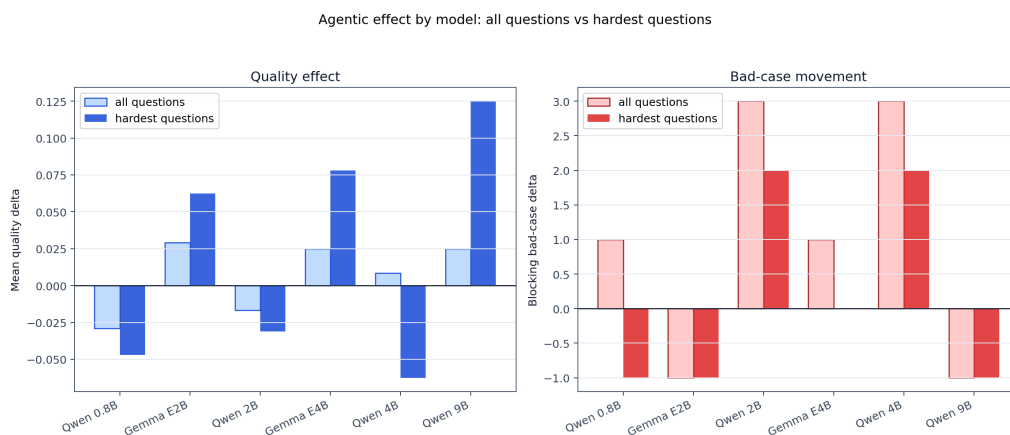


Figure 10: Agentic effects by model on blocking bad-case changes under ICU Bench 30.

We want to compare whether agentic mode improves answer quality generally across models and whether it changes the number of bad cases. However, the result does not show a universal pattern that agentic mode is always better. Among these models, Gemma 4 E2B, Gemma 4 E4B, and Qwen3.5 9B show quality improvement in agentic mode, but Qwen3.5 0.8B and Qwen3.5 2B become worse. On the blocking-case side, Qwen3.5 2B and Qwen3.5 4B show more blocking cases in agentic mode, while Gemma 4 E2B and Qwen3.5 9B show fewer blocking cases. This suggests that agentic retrieval may help stronger or more stable models, but it is not automatically helpful for every small model.

7 Bad Case Analysis and Discussion

7.1 Why the 0.8B model fails

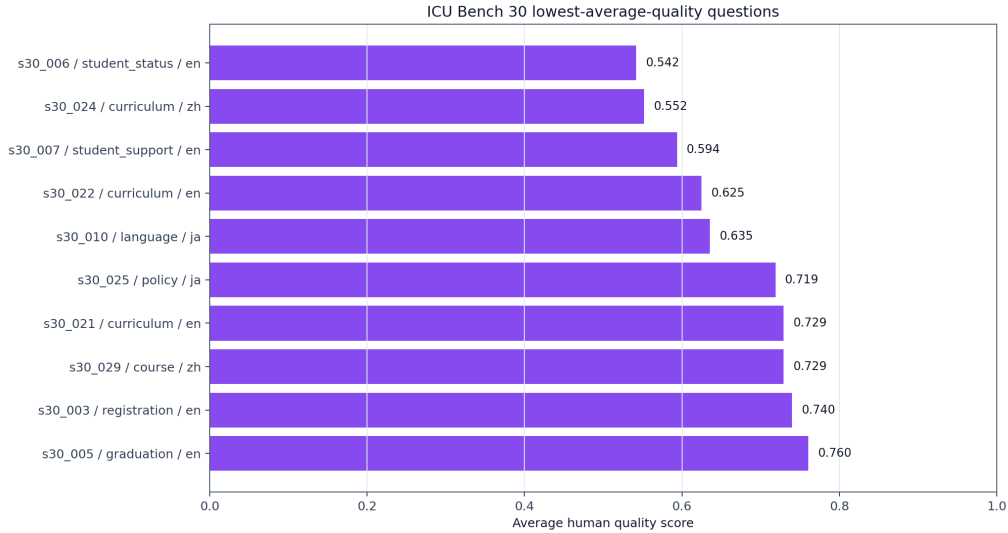


Figure 11: ICU Bench 30 lowest-average-quality questions.

7.2 Why average scores hide deployment risk

Small models’ average scores can get close to larger models’ scores. However, this does not necessarily suggest that they are capable enough to be deployed in a real situation. Average scores capture average-case usefulness, while answer consistency and blocking cases strongly affect user experience and can cause potential risk by providing unusable answers. Therefore, in our consideration, the smallest high-scoring model is not necessarily the smallest reliable model.

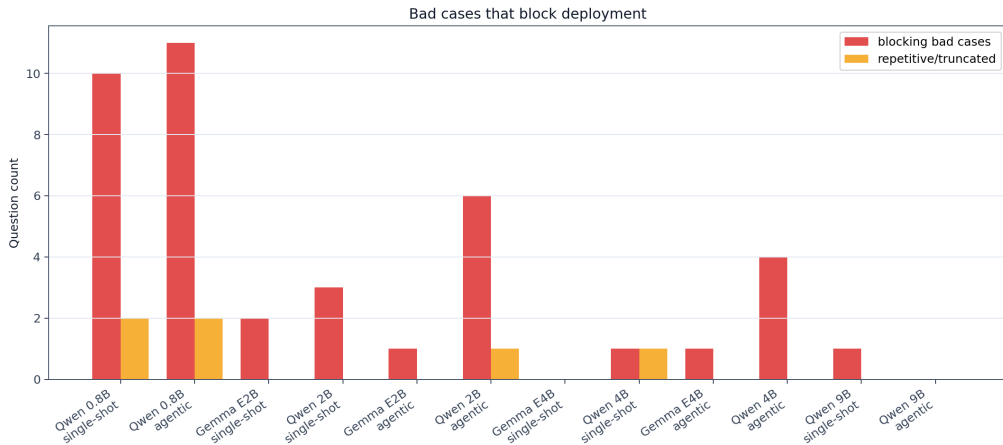


Figure 12: ICU Bench 30 blocking bad cases by model and answer mode.

7.3 Feasibility boundary for local university QA

In certain narrow cases for university QA, small models have the potential to be workable when there is a strong retrieval-and-orchestration layer outside the model. However, the real boundary is not whether the small model can answer most questions, but whether it can avoid certain catastrophic answer failures. Therefore, average quality may saturate earlier than reliability.

8 Conclusion

8.1 Main findings

Although small models can reach a high average score that is close to larger models, we need to add blocking-case screening to avoid catastrophic answer failures. In this situation, we found that the smallest capable model is not necessarily the smallest high-scoring model.

8.2 Limitations

Although we used two SOTA-level LLM judges to evaluate generated answers, these metrics are not based on real human validation. Therefore, future work needs real human evaluation and validation to see how human judgments align with the evaluation by the LLM judges.

8.3 Future work

References

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., and Zhu, C. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, Singapore, 2023. Association for Computational Linguistics.

A Appendix